CS 550 Final Project Proposal: Replication of the Hawkeye and Mockingjay Cache Replacement Policies

Vito Wang, Endong Dai, and Xiaotian Tong

***Abstract*—This project replicates two cache replacement policies that learn from Belady's optimal replacement algorithm, Hawkeye [1] and Mockingjay [2], in gem5 and evaluates them on SPEC CPU2017. Hawkeye reconstructs the decisions that the optimal policy would have made on past accesses and learns which lines to keep. Mockingjay instead predicts when each line will be reused and evicts the line reused furthest in the future. We will validate each policy with logs and microbenchmarks, and compare their miss rate and performance with each other and with the policies that gem5 already provides, such as LRU, DRRIP, and SHiP.**

I. Introduction & Problem Statement

The last-level cache (LLC) is the last on-chip structure that can hide the long latency of DRAM, so every LLC miss stalls the processor for hundreds of cycles. When the LLC is full, the replacement policy chooses which line to evict, and this choice largely decides the miss rate. The common least-recently-used (LRU) policy works well when recently used data is soon reused. However, it fails when a program cycles through a working set larger than the cache, because every line is then evicted just before it is reused, and when streaming data that is never reused pushes useful lines out. Improved policies handle some of these patterns, for example by giving new lines a low priority so that they leave the cache quickly unless they are reused (RRIP [3]), or by learning which load instructions bring in lines that are reused (SHiP [4]). However, each is built around a particular class of access patterns.

Belady's algorithm [5], also called OPT, is the optimal policy. It evicts the line whose next use lies furthest in the future. OPT cannot be built in hardware because it needs future knowledge. Hawkeye [1] avoids this problem by looking at the past instead. It computes what OPT would have done on past accesses, learns which load instructions tend to bring in lines that OPT would keep, and uses this to guide future evictions. However, it only classifies each line as worth keeping or not. Its follow-up, Mockingjay [2], predicts how far in the future each line will be reused, which lets it order lines the way OPT does. The two policies were evaluated in the Cache Replacement Championship simulator and in ChampSim. gem5 includes neither of them, so they have not been evaluated with gem5's out-of-order CPU model.

This project aims to implement Hawkeye and Mockingjay in gem5 [6], [7]. The goal is to verify their functional correctness and to compare their miss rate and performance with each other and with the baseline policies in gem5.

II. Related Work

Classic replacement policies rank lines by recency or frequency, e.g., LRU and LFU [8]. RRIP [3] instead predicts how soon each line will be re-referenced, using a 2- or 3-bit counter per line, and inserts new lines with a long predicted interval so that data that is never reused leaves the cache quickly. DRRIP chooses between two RRIP variants at run time through set dueling [9], which compares the policies on a few dedicated sets. A previous CS 550 project evaluated DRRIP, SRRIP, and BRRIP against LRU in gem5. We use these policies as baselines.

Other policies predict reuse for each load instruction. SHiP [4] learns, for each program counter (PC), whether the lines it inserts are reused, and SDBP [10] predicts dead blocks from a small sampler. Both learn from the behavior of a heuristic policy, so they inherit its limits.

Hawkeye [1] and Mockingjay [2] instead learn from OPT itself. Hawkeye won the 2nd Cache Replacement Championship (CRC2) [11], and we will use its CRC2 code as a reference.

III. Research Questions

- **What are the Hawkeye and Mockingjay policies, and can they be implemented correctly in gem5?**
- **How do their LLC miss rate and IPC compare with each other and with LRU, DRRIP, and SHiP?**
- **Why can they reduce misses where LRU and RRIP fail, and when does predicting reuse distances help more than a keep-or-evict decision?**

IV. Proposed Solution

A. Hawkeye

Jain and Lin proposed Hawkeye [1], which has three parts.

OPTgen analyzes past accesses and reconstructs the decisions that OPT would have made. When a line is accessed again, OPTgen checks whether OPT could have kept the line in the cache since its previous access without exceeding the cache capacity. If so, OPT would have hit. Otherwise, OPT would have missed. To save storage, OPTgen observes only a small sample of the cache sets.

The Hawkeye Predictor learns from OPTgen which load instructions bring in lines that are worth keeping. It keeps a small counter for each program counter (PC). When OPTgen decides that OPT would have hit, the counter of the PC that last accessed the line goes up. Otherwise, it goes down. When

a line is accessed, the counter of its PC predicts whether the line is *cache-friendly* or *cache-averse*.

Replacement decides which line to evict based on these predictions. Cache-averse lines are evicted first. If a set has none, Hawkeye evicts the oldest cache-friendly line and lowers the counter of the PC that brought it in.

B. Mockingjay

Shah, Jain, and Lin proposed Mockingjay [2], which also has three parts.

The Sampled Cache records the recent accesses to a few sampled sets. When a line is accessed again, it measures the reuse distance, i.e., the number of accesses to the set since the line's previous access. A line that leaves the Sampled Cache without being reused is treated as never reused.

The Reuse Distance Predictor learns from these measurements the typical reuse distance of the lines brought in by each PC.

Replacement gives each line a counter of the estimated time remaining until its next use. The counter starts at the predicted reuse distance and counts down as the set is accessed. Mockingjay evicts the line furthest from its predicted reuse, and skips inserting a line that would be reused later than every line in the set.

V. Evaluation

A. Methodology

We will use gem5 v23.0.0.1 on the course AWS instance, in syscall-emulation (SE) mode with the x86 out-of-order CPU model (O3CPU). Following the original papers, the cache hierarchy has 32 KB L1 instruction and data caches, a 256 KB 8-way L2, and a 2 MB 16-way LLC. The course configuration has only two cache levels, so we will add the LLC. We will add the following components to gem5:

- **Two replacement policies**: new classes for gem5's replacement-policy interface, which passes the requesting packet, and thus the PC, on every insertion and hit, as the existing SHiP policy uses.
- **Per-line state**: the age or time remaining, and the PC, stored with each line because the eviction call does not see the packet.
- **Sampling and prediction**: OPTgen and the Hawkeye Predictor, and the Sampled Cache and the Reuse Distance Predictor of Mockingjay.
- **Bypass**: gem5's replacement policies cannot refuse an insertion, so Mockingjay will insert such a line as the next one to evict.
- **Logs and statistics**: a debug flag that prints each decision.

B. Functional Validation

The functional correctness of both policies will be tested in two ways.

a) Unit testing with logs: We will add logs to each component. For each access, the log will show OPTgen's decision or the measured reuse distance, the prediction, and the age or time remaining of each line. We will compare them with values calculated by hand for short access sequences and with values computed by a small script for longer ones.

b) Execution correctness: Each policy will be integrated into the LLC and will run a series of test programs. The replacement policy must not change program results, so the outputs are expected to match those under LRU. Some test programs will have known access patterns, such as an array that fits in the LLC and one slightly larger than the LLC, so that we can also check whether the miss rates match our expectations.

C. Performance Impact

Hawkeye and Mockingjay will be compared with each other and with baseline policies that gem5 already provides, such as LRU, DRRIP, and SHiP. We will focus on memory-intensive SPEC CPU2017 [12] benchmarks, where the LLC matters most, such as mcf, omnetpp, xalancbmk, and gcc. The comparison will focus on LLC misses per kilo-instruction (MPKI) and IPC. We will repeat it for one smaller and one larger LLC on mcf, omnetpp, and xalancbmk. We will also estimate the hardware cost from the storage of the added structures. We will evaluate a single core without hardware prefetching, so the multi-core and prefetch-aware versions of Mockingjay are out of scope.

VI. Milestones

The project is broken down into five milestones that span 5 weeks of work.

- **Milestone 1: Setup and Initial Research (Week 1):** We will add the LLC and the O3CPU to the gem5 configuration, run the baseline policies, study both papers and their code, and confirm that the PC reaches the LLC.
- **Milestone 2: Implementation (Week 2):** We will implement Hawkeye and Mockingjay in parallel.
- **Milestone 3: Functional Validation (Week 3):** We will run the unit tests and microbenchmarks and fix any differences from the expected behavior.
- **Milestone 4: Performance Evaluation (Week 4):** We will run SPEC CPU2017 with every policy and repeat the runs for two other LLC sizes.
- **Milestone 5: Final Evaluation and Reporting (Week 5):** We will analyze the results and write the final report and presentation.

VII. Division of Labor

Each of us will deliver part of the gem5 code. Vito and Endong will work on Hawkeye, including its implementation in gem5 and the microbenchmarks for its functional validation. Vito and Xiaotian will do the same for Mockingjay. Endong and Xiaotian will run the baseline and SPEC CPU2017 simulations, repeat them for the other LLC sizes, and collect and plot the statistics. All three will analyze the results and write the final report.

REFERENCES

[1] A. Jain and C. Lin, "Back to the future: Leveraging Belady's algorithm for improved cache replacement," in *Proc. ISCA*, 2016.

[2] I. Shah, A. Jain, and C. Lin, "Effective mimicry of Belady's MIN policy," in *Proc. HPCA*, 2022.

[3] A. Jaleel, K. B. Theobald, S. C. Steely Jr., and J. Emer, "High performance cache replacement using re-reference interval prediction (RRIP)," in *Proc. ISCA*, 2010.

[4] C.-J. Wu, A. Jaleel, W. Hasenplaugh, M. Martonosi, S. C. Steely Jr., and J. Emer, "SHiP: Signature-based hit predictor for high performance caching," in *Proc. MICRO*, 2011.

[5] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Systems Journal*, vol. 5, no. 2, pp. 78–101, 1966.

[6] N. Binkert *et al.*, "The gem5 simulator," *ACM SIGARCH Comput. Archit. News*, vol. 39, no. 2, pp. 1–7, 2011.

[7] J. Lowe-Power *et al.*, "The gem5 simulator: Version 20.0+," arXiv:2007.03152, 2020.

[8] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2019.

[9] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely Jr., and J. Emer, "Adaptive insertion policies for high performance caching," in *Proc. ISCA*, 2007.

[10] S. M. Khan, Y. Tian, and D. A. Jiménez, "Sampling dead block prediction for last-level caches," in *Proc. MICRO*, 2010.

[11] The 2nd Cache Replacement Championship (CRC2), held with ISCA 2017.

[12] Standard Performance Evaluation Corporation, "SPEC CPU2017," https://www.spec.org/cpu2017/.